UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 15

Paso por Referencia

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Objetivos de la sesión **2**

2 Prerrequisitos **2**

3 Secuencia de clase **2**
3.1 Apertura — La trampa invisible (15 min) 2
3.2 Paso por referencia y por apuntador (35 min) 3
3.3 Múltiples resultados y arreglos (15 min) 3
3.4 Bubble Sort (25 min) 4

4 Errores conceptuales frecuentes **4**

5 Soluciones a los ejercicios **5**
5.1 Ejercicio 1 — Elevar al cubo 5
5.2 Ejercicio 2 — Tres resultados en una pasada 5
5.3 Ejercicio 3 — Normalizar un arreglo 5
5.4 Ejercicio 4 — Bubble Sort descendente 6

6 Conexiones con temas posteriores **7**

7 Material de la sesión **7**

8 Rúbrica de ejercicios **7**

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Predecir si una función modificará o no el argumento original, dado su encabezado.
2. Declarar parámetros por valor, por referencia (`int&`) y por apuntador (`int*`).
3. Elegir el mecanismo de paso adecuado según el tipo de dato y la necesidad de modificación.
4. Usar `const&` para evitar copias costosas sin permitir modificaciones.
5. Implementar funciones que "devuelven" múltiples valores mediante referencias.
6. Implementar `swap` y explicar por qué necesita paso por referencia.
7. Completar la implementación de Bubble Sort con las tres funciones separadas.

90 minutos (sesión simple). Distribución sugerida:
15 min — apertura: la trampa invisible, demo en vivo
20 min — paso por valor vs referencia, diagramas de memoria
15 min — paso por apuntador, tabla comparativa, `const&`
15 min — múltiples resultados + arreglos
25 min — Bubble Sort: `swap` , `comparar` , `bubblesort` en vivo
Los ejercicios quedan como práctica para el cuaderno Jupyter.

Prerrequisitos

- Apuntadores, referencias y aritmética de apuntadores (sesión 14)
- *Array decay* y la pérdida del tamaño al pasar arreglos (sesión 14)
- Funciones: declaración, parámetros y valor de retorno (sesiones 11–13)

Los alumnos ya vieron referencias en la sesión 14 como *alias*. Esta sesión conecta ese concepto con el paso de parámetros, que es donde las referencias cobran utilidad práctica. Si algún alumno tiene experiencia en Python o Java, señala que en esos lenguajes los objetos se pasan por referencia automáticamente — el equivalente al `int&` de C++ lo hacen transparente el intérprete o la JVM.

Secuencia de clase

- **Apertura — La trampa invisible (15 min)**
  Escribe en el pizarrón (sin usar el cuaderno todavía):

```
void sumaUno(int n) { n = n + 1; }

int a = 10;
sumaUno(a);
cout << a;
```

Pide a los alumnos que predigan el resultado *por escrito* antes de ejecutarlo. Luego ejecuta. El objetivo es que la disonancia entre lo esperado (11) y lo obtenido (10) genere la pregunta; no expliques todavía.

> "¿Cuántos de ustedes esperaban 11?"
> Respuesta esperada: la mayoría levanta la mano.
> Respuesta correcta: 10. La función modifica su propia copia de `n`.
> Sigue con: "¿Por qué C++ hace esto? ¿No sería más intuitivo que sí cambiara?"
> Deja que discutan 2 minutos y luego presenta el modelo de memoria en el pizarrón.

Dibuja el diagrama de dos celdas separadas (la de `a` en `main` y la de `n` en `sumaUno`) con direcciones distintas. Solo entonces introduce la palabra "copia" y el término "paso por valor".

• Paso por referencia y por apuntador (35 min)

Muestra las tres versiones del `sumaUno` en secuencia (valor, referencia, apuntador) y pide que predigan el resultado de cada una antes de ejecutar.

> **&**
>
> Muchos alumnos escriben `sumaUno(&a)` cuando la función ya es por referencia (`int& n`). Esto es un error de compilación: `&a` es de tipo `int*`, no `int&`. El `&` en la declaración del parámetro hace que el compilador tome la dirección *implícitamente*. Enfatiza que el `&` va **solo en la declaración**, no en la llamada.

Presenta la tabla comparativa (valor / referencia / `const&` / apuntador) y el ejemplo de `const string&`. Conecta con `cin >>` vs `scanf &x` para anclar el concepto en algo que ya conocen.

> "Si `const&` es eficiente y seguro, ¿por qué no usarlo para todo?"
> Respuesta esperada: para tipos primitivos la dirección (`int&`) ocupa tanto como el dato mismo (4 vs 8 bytes en 64-bit), y la desreferencia añade una instrucción extra. El compilador puede optimizarlo, pero semánticamente es más claro `int` a secas cuando no se quiere modificar.

• Múltiples resultados y arreglos (15 min)

Demuestra la función `minMax` y pregunta: "¿Por qué `minVal` y `maxVal` son `int&` pero `arr` es `const int*`?" Deja que los alumnos respondan antes de explicar.

Cierra esta sección recordando el *array decay* de la sesión anterior: muestra que dentro de la función `sizeof(arr)` devuelve 8 (apuntador), no el tamaño real del arreglo. La relación directa con la sesión 14 refuerza la coherencia del curso.

- **Bubble Sort (25 min)**

  Construye el algoritmo en vivo, función por función:

1. Escribe `swap` y verifica que intercambia dos variables. Pregunta: "¿Qué pasaría si quitara el `&`?" (respuesta: intercambia copias, no cambia nada).
2. Escribe `comparar`. Pregunta: "¿Qué valor devuelve `comparar(3,5)`? ¿Y `comparar(5,3)`?" antes de ejecutar.
3. Escribe `bubblesort` y traza manualmente la primera pasada sobre el pizarrón con el arreglo de ejemplo `{33, 42, 88, 3, 50, 27, 77, 69}`.

Al terminar el Bubble Sort ascendente, cambia el `>` por `<` en `comparar` sin tocar ninguna otra función. Ejecuta y muestra que el arreglo queda en orden descendente. El punto: encapsular el criterio de comparación en una función separada hace el código flexible. Conecta con `std::sort` y los comparadores lambda.

Errores conceptuales frecuentes

1. "¿Cuál es la diferencia real entre referencia y apuntador si los dos modifican el original?"
La referencia es un alias que no puede ser nula ni reasignada; el apuntador es una dirección que puede ser `nullptr` y puede apuntar a distintos objetos. La referencia es más segura; el apuntador, más flexible. En paso de parámetros, preferir referencia salvo que se necesite `nullptr` o aritmética de apuntadores.

2. Confundir `int&` (referencia) con `&x` (dirección-de)
Un ejercicio útil: dar el mismo símbolo en tres contextos y pedir que identifiquen el significado: `int& r = x;` (declaración de referencia), `cout << &x` (dirección de `x`), y `int* p = &x;` (se asigna la dirección de `x` a `p`).

3. Pensar que los arreglos se copian al pasar a funciones
Viene de Java/Python donde los arrays sí se pueden copiar con facilidad. Demostrar modificando el arreglo dentro de una función y mostrando que el original cambió.

4. Escribir `swap` sin variable temporal
Algunos alumnos intentan `a = a + b; b = a - b; a = a - b;` o versiones con XOR. Funcionan para enteros pero fallan para flotantes o tipos no numéricos. La versión con temporal es la correcta y la que usa `std::swap` internamente.

5. Olvidar `n - i - 1` en el límite interno del Bubble Sort
El índice interno puede ser `n - 1` y el algoritmo sigue siendo correcto, solo hace comparaciones redundantes al final (los últimos `i` elementos ya están ordenados). No es un error grave, pero es una oportunidad para hablar de optimización.

Soluciones a los ejercicios

• Ejercicio 1 — Elevar al cubo

```
void cubo(int& n) {
    n = n * n * n;
}

int arr[4] = {2, 3, 4, 5};
for (int i = 0; i < 4; i++)
    cubo(arr[i]);
// arr = {8, 27, 64, 125}
```

Algunos alumnos intentan `n = n^3` (XOR bit a bit, no potencia). Recordar que C++ no tiene operador de exponenciación; se usa `pow(n, 3)` de `<cmath>` o la multiplicación explícita.

• Ejercicio 2 — Tres resultados en una pasada

```
void estadisticas(const int* arr, int n,
                  int& minVal, int& maxVal, int& suma) {
    minVal = maxVal = arr[0];
    suma = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < minVal) minVal = arr[i];
        if (arr[i] > maxVal) maxVal = arr[i];
        suma += arr[i];
    }
}

int datos[7] = {12, 5, 8, 21, 3, 17, 9};
int mn, mx, s;
estadisticas(datos, 7, mn, mx, s);
cout << "min=" << mn << " max=" << mx << " suma=" << s <<
    endl;
// min=3 max=21 suma=75
```

• Ejercicio 3 — Normalizar un arreglo

```
void normalizar(double* arr, int n) {
    double maxVal = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > maxVal) maxVal = arr[i];

    for (int i = 0; i < n; i++)
        arr[i] /= maxVal;
}

double v[5] = {20.0, 50.0, 10.0, 80.0, 35.0};
normalizar(v, 5);
// v = {0.25, 0.625, 0.125, 1.0, 0.4375}
```

Punto de discusión: ¿qué pasa si `maxVal` es cero? División por cero. Para un arreglo de datos reales habría que protegerse contra ese caso. Buen gancho para hablar de validación en los límites del sistema.

• Ejercicio 4 — Bubble Sort descendente

Solo hay que cambiar una línea en `comparar`:

```
// Ascendente:
bool comparar(int a, int b) { return a > b; }

// Descendente (unica linea que cambia):
bool comparar(int a, int b) { return a < b; }
```

La explicación: `bubblesort` intercambia `arr[j]` y `arr[j+1]` cuando `comparar` devuelve `true`. Si la condición de intercambio es "el izquierdo es menor", los elementos pequeños migrarán hacia el final (orden descendente).

Pregunta abierta para los alumnos más avanzados: "¿Podrías hacer que `comparar` sea un *parámetro* de `bubblesort`, para que el usuario decida el criterio de ordenamiento?"

```
void bubblesort(int* arr, int n, bool(*cmp)(int,int)) {
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-i-1; j++)
            if (cmp(arr[j], arr[j+1]))
                swap(arr[j], arr[j+1]);
}

bubblesort(arr, n, comparar);
```

Este es el germen del polimorfismo de comportamiento, que en C++ moderno se expresa con lambdas y `std::function`. No es necesario profundizar ahora,

> pero despierta curiosidad hacia los temas futuros.

Conexiones con temas posteriores

- **Sesiones 16–17** — Recursividad: las funciones recursivas se llaman a sí mismas con paso por valor o por referencia según si necesitan acumular resultados.
- **Sesiones 18–20** — Memoria dinámica: `new` devuelve un apuntador que se pasa entre funciones; entender paso por apuntador es requisito.
- **Sesiones 21–25** — Orientación a Objetos: los métodos reciben `this` (apuntador al objeto) implícitamente; los constructores de copia usan `const T&`.
- **C++ moderno** — `std::sort` con lambda, `std::swap`, `std::move` (referencias rvalue, C++11) son extensiones directas del paso por referencia.

Material de la sesión

- **Cuaderno:** `sesiones/sesion15_paso_por_referencia.ipynb`
- **Ejemplos:** `materialInicial/sesion15/referencias01-06.cpp`, `sort.cpp`
- **Notas alumno:** `notas_alumno/pdf/sesion15_paso_por_referencia.pdf`

Rúbrica de ejercicios

Ejercicio / Criterio	**Puntos**	**Evidencia**
Ejercicio 1 — Elevar al cubo		
Parámetro declarado como `int&`	4	firma de la función
Resultado correcto para los 4 valores	4	salida esperada
No usa `pow`	2	multiplicación explícita
Ejercicio 2 — Tres resultados		
Tres parámetros de salida como `int&`	3	firma correcta
Una sola pasada (un solo `for`)	4	código con 1 ciclo
Resultados correctos	3	min=3, max=21, suma=75
Ejercicio 3 — Normalizar		
Arreglo como `double*`, modifica el original	3	sin `const` en `arr`
Encuentra el máximo correctamente	3	primera pasada
Divide todos los elementos	4	segunda pasada
Ejercicio 4 — Bubble Sort descendente		
Identifica la línea correcta a cambiar	5	solo `comparar` modificado
Explicación escrita correcta	5	razonamiento con criterio de intercambio
Total	**40**	